Data Lakehouse - Michaël Bettan

Data Lake vs Warehouse vs Lakehouse

- **Data Lake:** Stores raw data in various formats at low cost but lacks structure, governance, and performance for complex queries
- **Data Warehouse:** Stores structured data optimized for fast querying and BI use cases but can be expensive and inflexible for diverse data types. They traditionally use schema-on-write and offer robust management features like ACID transactions and indexing.
- **Data Lakehouse:** Combines the best of both worlds, offering low-cost storage, diverse data types, and excellent performance with robust governance.
    - **Solves data lake challenges** (data duplication, inconsistent data, lack of schema enforcement).
    - **Solves data warehouse challenges** (rigidity, difficulty in making changes).
    - **Single processing path:** Spark handles both batch and streaming incrementally (replaces dual-codebase Lambda architecture).

The Lakehouse architecture leverages cloud object storage combined with open standards like Delta Lake to provide ACID transactions and robust governance. This approach supports record-level updates and eliminates the complexity of separate lake and warehouse systems

- **Spark:** Acts as the primary processing engine, unifying real-time and batch workloads under a single, incremental computational path.
- **Delta Lake:** An open storage layer utilizing Parquet files and a transaction log (stored in _delta_log/) to provide ACID integrity, schema enforcement, and time travel across cloud-native object storage.

Medallion architecture

Uses zones to manage data quality and usability:

- **Source:** This is where data originates, directly from various systems like databases, files, or event streams. Data here is in its raw form and can have varying levels of quality and governance.
- **Bronze:** The first zone within the lakehouse, data is ingested here as is, generally in Delta Lake format. It may contain duplicates, especially from transactional systems. Data is appended, and can retain history. Full data loads replace or are appended depending on cost concerns and the presence of identifiers. The primary goal is to streamline the loading of the silver layer.
- **Silver:** Data in this zone is cleaned, quality rules applied, and duplicates are removed through merging (based on unique identifiers). If identifiers and ordering information are absent, complete replacement of data occurs. Silver contains data that businesses use to make decisions.
- **Gold:** This zone contains curated and optimized data that is specifically structured to answer business questions. Gold tables are created through aggregation, filtering, joins, and other transformations. They are highly tuned for reading, often supporting reports and dashboards.

Lambda architecture

The Lambda architecture attempts to balance the need for both low-latency and high-throughput data processing. It does this by implementing two processing paths:

- **Speed Layer (Hot Path):** Processes data in real-time or near real-time using stream processing technologies (e.g., Apache Storm, Apache Flink, Apache Kafka Streams). This layer focuses on providing quick, albeit potentially inaccurate, results on recent data. Accuracy is sacrificed for speed as there might not be time for complex calculations or to incorporate all available data.
- **Batch Layer (Cold Path):** Processes data in batches using batch processing frameworks (e.g., Apache Hadoop, Apache Spark). This layer handles historical data and performs more comprehensive, accurate computations. The results from the batch layer eventually correct or enhance the initial results from the speed layer.
- **Serving Layer:** Combines and serves the results from both the speed and batch layers to provide a unified view of the data.

Challenges with Lambda Architecture:

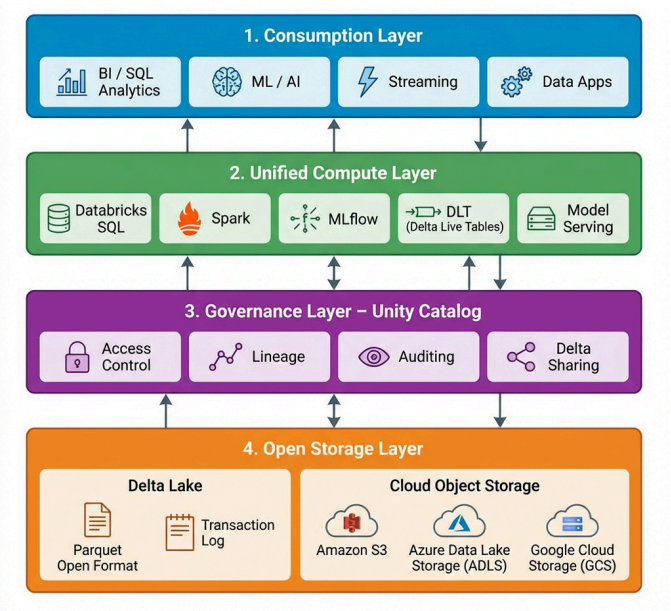
- **Complexity**: Maintaining two separate codebases (one for real-time and one for batch) is complex and expensive. It requires expertise in different technologies and increases development and operational overhead.
- **Data Duplication**: The same data is processed twice, leading to data duplication and increased storage costs.
- **Inconsistency**: Reconciling results from two different processing paths can be challenging and might lead to temporary inconsistencies in the data.

A lakehouse architecture simplifies data processing by employing a single processing path using technologies like Apache Spark. This single path handles both real-time and batch processing. The key here is the concept of incremental processing and Spark's ability to handle both micro-batches and large batch jobs.

Core Principles

- **Open Formats:** Leverages open standards like Apache Parquet and Delta Lake to prevent vendor lock-in and enable cross-engine access.
- **ACID Transactions:** Ensures data integrity on cloud object storage through robust transactional guarantees provided by the Delta Lake transaction log.
- **Schema Enforcement & Evolution:** Protects data quality by preventing incompatible writes while supporting governed structural changes over time.

- **Unified Governance:** Standardizes security and discovery for all data and AI assets via a centralized model like Unity Catalog.
- **BI + ML on same data:** Enables both business intelligence and machine learning directly on the lakehouse, eliminating redundant data silos and costs.
- **Decoupled Storage & Compute:** Scales storage and compute resources independently to optimize the total cost of ownership and performance.
- **Streaming + Batch unified:** Unifies the ingestion and processing of real-time and batch workloads using identical APIs and Delta tables.



Architecture

1. **Ingestion Layer:** Data from various sources (databases, APIs, IoT devices, etc.) is ingested into the lakehouse in its raw format.
2. **Storage Layer:** This layer uses open, standardized formats like Parquet, ORC, Avro, or Delta Lake to store data efficiently and cost-effectively in cloud storage. Delta Lake is particularly popular due to its ACID properties.
3. **Metadata Layer:** A crucial component that provides a catalog of data assets, including schemas, data lineage, and other metadata, improving data discovery and governance. This often utilizes a metastore, such as Unity Catalog from Databricks.
4. **Processing Layer:** This is where data is processed and transformed using tools like Spark, Trino, or other query engines.
5. **Consumption Layer:** This is where users access and utilize the processed data for BI, analytics, or ML applications.

Table Format (Delta Lake)

Open-source storage layer that provides ACID transactions and scalable metadata handling for data lakes. It is often used with Apache Spark and is optimized for both batch and streaming data.

- **File Format:** Leverages Parquet files coupled with a JSON-based transaction log stored in the _delta_log/ directory.
- **ACID Transactions:** Guarantees full transactional integrity on cloud object storage, ensuring partial writes are never visible to readers.
- **Schema Enforcement:** Protects data quality by automatically rejecting incompatible writes at the time of ingestion.
- **Schema Evolution:** Supports governed structural changes via mergeSchema or overwriteSchema options without compromising data integrity.
- **Time Travel:** Enables access to historical data snapshots using specific version numbers or timestamps.
- **MERGE (Upsert):** Performs atomic insert, update, and delete operations, providing a critical foundation for CDC pipelines.
- **File Immutability:** Preserves original files by generating new Parquet files for every write operation.
- **OPTIMIZE:** Enhances read performance by compacting small files into larger, standardized Parquet files.
- **VACUUM:** Permanently removes unreferenced data files, adhering to a default retention period of 7 days.
- **Z-Ordering:** Co-locates related data within files to maximize the efficiency of data skipping during queries.
- **Liquid Clustering:** Provides a dynamic alternative to partitioning and Z-Order by automatically reorganizing files based on clustering keys.
- **Deletion Vectors:** Utilizes a soft-delete mechanism to mark deleted rows, enabling row-level concurrency without immediate file rewrites.

Table Format (Iceberg)

Open table format providing warehouse-grade features on data lakes. Engine-agnostic — readable by Spark, Trino, Flink, Hive, and others.

- ***ACID Properties**: Full ACID via snapshot isolation — concurrent operations handled safely*
- ***Schema Evolution**: Add, remove, rename columns without rewriting data*
- ***Hidden Partitioning**: Manages partition layout automatically — no manual directory structure management*
- ***Time Travel**: Query past snapshots via snapshot ID or timestamp*
- ***Metadata layer tracks**: table schema, data file locations, partitioning info, snapshot history*
- ***Multi-engine**: Same Iceberg table readable by multiple compute engines simultaneously*

Table Format (Uniform)

- **Universal Format (UniForm):** Enables a single Delta table to be read simultaneously as Delta, Iceberg, or Hudi without copying data.
- **Interoperability:** Eliminates multi-format fragmentation, allowing different teams and compute engines to access the same data source.

- **Table Configuration:** Enabled at creation via TBLPROPERTIES ('delta.universalFormat.enabledFormats' = 'iceberg').

ACID Transactions

- **Atomicity:** Guarantees that a write operation either fully succeeds or fully fails, ensuring that partial writes are never visible to readers.
- **Consistency:** Protects table integrity through schema enforcement, preventing corrupt or incompatible data from being ingested.
- **Isolation:** Ensures that concurrent readers and writers do not block each other, managed via Optimistic Concurrency Control (OCC).
- **Durability:** Permanently records every committed transaction in the Delta Log to ensure data survives system failures.

Databricks Platform Architecture

	Control Plane	***Data Plane***
Hosting	*Managed within the Databricks cloud subscription*	*Resides within the customer's own cloud subscription*
Components	*Includes Web UI, Cluster Management, Workflows, and Notebooks*	*Consists of compute clusters and storage (DBFS, Delta tables)*
Management	*Fully administered by Databricks*	*Administered by the customer via Databricks tooling*

Metadata Management

Metadata is managed at two distinct levels: table-level via the **Delta Transaction Log** and platform-level via **Unity Catalog**.

Delta Transaction Log (Table-Level)

- **Log Directory:** Every Delta table maintains a _delta_log/ directory consisting of ordered JSON commit files.
- **Commit Metadata:** Each commit records essential information including schema, file locations, min/max column statistics, and the operation type.
- **Checkpoint Files:** Created every 10 commits to optimize performance by avoiding a full replay of the log history during reads.
- **Read Flow:** The engine reads the last checkpoint and then applies only the Delta logs committed after that specific version.
- **Source of Truth:** The log is the table, acting as the single source of truth for all read and write operations.

Unity Catalog (Platform-Level)

- **Regional Metastore:** A single metastore per region is shared across all workspaces, replacing fragmented per-workspace Hive Metastores.
- **Namespace Hierarchy:** Utilizes a 3-level namespace consisting of Catalog → Schema → Table / View / Volume / Function.
- **Unified Governance:** Centralizes control for Delta tables, views, ML models, dashboards, files (Volumes), and SQL functions.
- **Automated Lineage:** Captures column-level lineage automatically across notebooks, jobs, and DLT pipelines without manual configuration.
    - **system.access.audit:** All data access and administrative actions
    - **system.billing.usage:** DBU consumption per workload type
    - **system.access.column_lineage:** Column-level lineage graph
    - **system.access.table_lineage:** Table-level lineage graph